Setting Up R Working Environments

A 4-Step System for Reproducible Research

Ethiopian Statistics Service (ESS)

September 18 to 20, 2026

Step 1: RStudio Projects

The Foundation of Your Workflow

Setup: Create Project Folder

- Create a folder with subfolders:
  - `my_project/`
    - `data/` - Raw and processed data
    - `scripts/` - R code (cleaning, analysis, plotting)
    - `output/` - Figures, tables, reports
    - `doc/` - Notes, papers, documentation

Initialize the Project

- `File > New Project > Existing Directory`
- **Key Benefit:** Working directory auto-set to project root
- **Example:** `read.csv("data/raw_data.csv")` always works

Step 2: Reproducibility with `renv`

Prevent "It works on my machine!" Problems

What is `renv`?

- Creates an isolated package library per project
- Tracks exact package versions in `renv.lock`
- Ensures everyone uses the same package versions

Workflow Example

1. `renv::init()` - Initialize renv
2. `renv::install("dplyr")` - Install packages
3. `renv::install("ggplot2")`
4. `renv::snapshot()` - Save versions to `renv.lock`

Result: `renv.lock` tracks exact package versions

Step 3: Structuring Your Analysis

R Scripts vs R Notebooks

R Scripts (.R) - Repeatable Tasks

- `scripts/01_load_data.R`
  - `library(dplyr)`
  - `raw_data <- read.csv("data/raw_data.csv")`
- `scripts/02_clean_data.R`
  - `clean_data <- raw_data %>%`
  - `filter(!is.na(value)) %>%`
  - `mutate(date = as.Date(date))`

R Notebooks (.Rmd) - Exploratory Analysis

- Combines narrative, code, and output
- Knit to HTML, PDF, or Word
- **Pro Tip:** Use `source("scripts/01_load_data.R")`

Step 4: Advanced Customization with `.Rprofile`

Automate Your R Startup

What is `.Rprofile`?

- A script that runs **every time R starts**
- Common uses:
  - Set default CRAN mirror
  - Auto-load frequently used packages
  - Define helper functions
  - Set memory limits

Example `.Rprofile`

```
options(repos = c(CRAN =
"https://cloud.r-project.org"))
if (requireNamespace("tidyverse")) {
  library(tidyverse) }
clean_names <- function(df) {
  names(df) <- tolower(gsub(" ", "_", names(df))) }
```

Real-World Scenario: Team Collaboration

Why This Setup Saves Headaches

Without renv

- Alice has dplyr 0.8.5
- Bob has dplyr 1.1.0
- `across()` doesn't work for Alice
- %>% deprecated warning for Bob
- **Result:** Hours wasted debugging

With renv

- Both use `renv::restore()`
- Both get **exactly** dplyr 1.0.10
- Identical package versions
- Code runs the same everywhere
- **Result:** Focus on analysis, not debugging

Ultimate Takeaway

RStudio Project + `renv` =

Portable, Isolated, and Reproducible Research

Set it up once, save yourself from future headaches!